

SpecSlice 白皮书

非侵入式 AI 编码意图层

用证据图回答:这段代码是干什么的,谁能证明。

功能全景 · 设计哲学 · 算法要点 · 质量体系 · 路线评估

2026 年 6 月 · 与仓库同步演进

非侵入式 AI 编码意图层 —— 用证据图回答"这段代码是干什么的,谁能证明"。

本文是 SpecSlice 的功能全景、设计哲学、算法要点与路线评估的权威中文文档。

命令行为以 `specslice --help` 与真实输出为准;本文随版本演进更新。

0. 一分钟读懂 SpecSlice

它是什么:一个命令行工具。把你的代码仓库(代码、文档、测试、SQL、路由)扫描成一张证据图,存进仓库里的一个 SQLite 文件(`.specslice/graph.db`),然后在这张图上回答各种问题。

它能回答什么:

你想知道	用哪个命令
"下单逻辑在哪?"(中文也行)	<code>specslice search 下单</code>
"改了这两个文件,会影响哪些需求和测试?"	<code>specslice impact</code>
"这个接口往下调了谁?最后写哪张表?"	<code>specslice trace /api/orders</code>
"哪些代码没人调用,可以删?"	<code>specslice dead-code</code>
"这个仓库有哪些业务模块?"	<code>specslice propose</code>
"文档里写的代码路径还存在吗?"	<code>specslice check</code>

它不做什么:不改你的任何一行代码、文档、测试(全部状态只写在 `.specslice/` 里,删掉即彻底清除);不上传任何数据(零网络);不内置任何 AI 模型(它产出"证据 + 提示词",由你自己的 AI 客户端消费)。

给谁用:接手陌生仓库的工程师、做代码迁移/重写的团队、想给 AI Agent 喂准确上下文的人。

0.1 五分钟上手

```
# 1. 在你的仓库根目录初始化(自动检测语言,生成配置)
specslice init

# 2. 建立索引(首跑数秒到数十秒,重跑增量)
specslice index

# 3. 开始提问
specslice search 支付回调      # 中文/英文都可以
specslice impact              # 当前分支改动影响了什么
specslice dashboard           # 生成一个离线 HTML 总览面板
```

0.2 关键术语(读懂本文只需这五个)

术语	含义
节点 (Node)	图里的一个"东西":一个函数、一个文件、一段文档、一个测试、一张数据库表、一条 HTTP 路由
边 (Edge)	两个节点的关系:A 调用 B、文档 D 描述了代码 C、测试 T 验证了函数 F
证据 (Evidence)	每条边背后的出处:具体到文件、行号、调用点 —— 不是"AI 觉得像"

术语	含义
候选 (Candidate)	AI 根据图提出的"这段代码属于某业务"的待确认猜想
确认 (Confirm)	人审核候选后落库,从此成为权威事实 —— AI 提议、人拍板

1. 定位

大多数"代码智能"工具回答 "这个符号在哪";SpecSlice 还要回答 "这段代码是为了什么业务存在的,证据是什么"。

SpecSlice 把仓库索引成一张 SQLite 证据图:

- 节点:符号(函数/类/方法…)、文件、文档章节、需求、测试、HTTP 路由、数据库表……
- 边:calls / references / implements / verifies / persists / documents……每条边携带证据(调用点、文档链接、测试引用)与置信度。

在这张图之上提供:代码搜索、影响面分析、死代码检测、行为事实抽取、业务模块证据包,以及"AI 提议 → 人确认"的业务逻辑沉淀 workflow。

1.1 四条设计公理

1. 非侵入(零写回)。绝不修改、注释、提交用户源码;一切状态在 `.specslice/` 下,可随时删除重建。
2. 证据优先于断言。每条边背后是具体事实,不是黑盒启发;AI 输出永远附带证据链。
3. AI 提议、人确认。业务候选由代码/文档/测试事实生成,经人审后才成为权威。
4. 通用性优先于特例。所有启发式必须是跨语言、跨仓库形态的通用规则,用真实大仓回归验证;禁止为单一仓库写特判。

1.2 与同类工具的差异

能力	grep/ctags	LSP/IDE	GitNexus / CodeGraph 类	SpecSlice
符号定位	✓	✓	✓	✓
跨文件调用图	×	部分	✓	✓(分层:结构层 + 可选 SCIP 精确层)
文档 ↔ 代码漂移	×	×	×	✓(check)
业务模块自动划分	×	×	弱(目录罗列)	✓(图社区 + 作者目录共识命名)
行为事实(分支/纯度/常量/契约)	×	×	×	✓(P24 套件)
移植/重写追踪	×	×	×	✓(port-coverage / graph-equiv)
非侵入保证	✓	✓	不一	✓(公理)

2. 架构

```
crates/
├── specslice-core    # 图领域模型:Node / Edge / Evidence / ArtifactId / NodeKind
├── specslice-store    # SQLite 存储 + 迁移(.specslice/graph.db),FTS5 全文层
├── specslice-engine   # 索引器、扫描器、搜索、全部分析(大脑)
├── specslice-lang-dart # Dart analyzer sidecar(领域感知:Riverpod/Hive/导航/IAP)
├── specslice-cli      # specslice 二进制(31 个子命令)
└── specslice-mcp      # specslice-mcp:Model Context Protocol 服务器(7 个工具)
```

2.1 索引管线

```
init(语言检测,生成 .specslice.yaml)
├── index
│   ├── docs 扫描(markdown/需求/ADR → doc_section / requirement 节点)
│   ├── 结构层:tree-sitter 统一通道(并行解析 + 单文件解析预算)
│   │   ├── Rust / TypeScript / Python / Go / Java / C / C++ / Swift
│   │   └── / C# / Ruby / PHP / Kotlin
│   ├── Dart analyzer sidecar(可选)
│   ├── SCIP 精确层(可选 enrichment;增量:源未变直接复用 .scip)
│   ├── 语义边:routes / persists / verifies / documents
│   ├── FTS5 全文层(代码体 + 文档注释 + markdown,CJK bigram)
│   └── 单事务批量提交(WAL,checkpoint 合并)
```

关键工程决策:

- 结构层先行,精确层叠加:SCIP/LSP 只把精确 calls/references 绑定到已存在的符号上;缺失外部 indexer 是一条清晰的"仅结构"提示,而非错误。
- 解析预算:单文件解析超时即降级跳过,故意构造的坏语法 fixture 海(如 TypeScript 编译器测试集)不会拖死索引。
- 批量写入:bulk upsert + 显式事务 + WAL autocheckpoint 暂停,大仓索引的 I/O 开销被压缩到秒级。
- 语言检测防误判:语言入选需要 manifest 佐证(pubspec/package.json/Cargo.toml/go.mod...),或 ≥ 3 个源文件 / $\geq 25\%$ 占比 —— 单个 gdb 脚本不会让 Rust 仓库多出一个 python 通道(rust-analyzer 实测回归)。

2.2 性能基准(Apple Silicon,实测)

仓库	语言	规模	冷索引	稳态(增量)
Redis	C	~20 万行	~11s	秒级
TypeScript 编译器	TS	2 万+ 文件	~28s	—
Spring Framework	Java/Kotlin	~7600 文件	~10s	—
rust-analyzer	Rust	~1500 文件	46s(含 SCIP 首跑)	5.5s

仓库	语言	规模	冷索引	稳态(增量)
Laravel / Rails / OkHttp / Jellyfin	PHP/Ruby/Kotlin /C#	大型	秒级~十秒级	—
SpecSlice 自身(自举)	Rust	6 crate	~15s	秒级

SCIP 增量:`.scip.inputs` 摘要未变即整语言复用上次输出,典型增量场景从 43s 降到 ~2s。

3. 功能全景(31 个 CLI 命令 + 7 个 MCP 工具)

3.1 建立与维护

命令	作用
<code>init</code>	检测语言、生成 <code>.specslicel.yaml</code> + <code>.specslicel/graph.db</code>
<code>index</code>	索引文档+代码入图;重复执行增量
<code>export</code>	导出图为可携带 bundle
<code>stats</code>	命令使用统计账本(<code>.specslicel/stats.jsonl</code>)

3.2 导航与检索

命令	作用
<code>search</code>	代码图搜索:结构评分(id/名称/路径/证据/邻接)+ BM25 全文内容层,中英双语(CJK bigram),测试/工具/示例路径降权,每个命中附 grounding 源码片段
<code>trace</code>	接口 → 整张图:沿 calls/references/persists 做前向传递闭包,输出 controller→service→SQL→table 完整下游链与触达表汇总
<code>graph</code>	图渲染:JSON / Mermaid / 自包含 HTML(离线可开)
<code>context</code>	需求 → agent-ready 上下文包
<code>slice</code>	需求 → 文档/实现/测试切片

3.3 变更影响

命令	作用
<code>impact</code>	git diff → 受影响的需求/文档/测试; <code>--worktree</code> 支持未提交变更;输出 text/json/mermaid
<code>select-tests</code>	diff → 应运行的测试清单(带原因/置信度,不自动执行)
<code>graph-diff</code>	两份 graph.db 快照对比(CI artefact 场景)

3.4 质量与一致性

命令	作用
<code>check</code>	一致性检查:文档过期引用(doc_stale_code_ref)、孤儿需求 + 图建议实现、断链、缺失关联测试
<code>dead-code</code>	入口点不可达符号报告(带原因/置信度,绝不自动删除)

命令	作用
<code>similar</code>	结构指纹比对,报告结构相同的函数/方法簇
<code>questions</code>	图中需要人/Agent 澄清的事实清单

3.5 行为事实(P24 移植/重构套件)

命令	作用
<code>facts</code>	每个符号体的分支/循环/return/比较/空值/抛出/await 计数 + 决策证据行 + 纯度
<code>purity</code>	pure/impure/unknown 普查 + 副作用原因(io/async/ui/time/randomness/global_mutation),沿调用图传播
<code>constants</code>	字面量目录:按值聚合全部 int/float/string/bool/char,定位每个魔法值
<code>contract</code>	数据契约:字符串里的 CREATE TABLE schema + <code>obj['key'] ?? default</code> 序列化键映射
<code>suggest-tests</code>	由行为事实生成确定性单测建议(分支多者优先,不写测试)
<code>feature-pack</code>	一键切片导出:特性所需全部上下文(符号+事实+依赖边+常量+契约+测试建议)打成自包含 JSON

3.6 移植与重写追踪

命令	作用
<code>port-coverage</code>	按符号名对比源/目标 graph.db:已移植/缺失/目标独有 + 按文件覆盖率
<code>route-coverage</code>	按规范化路由对比客户端/服务端 graph.db:已服务/缺失/独有
<code>graph-equiv</code>	同一业务切片在两图中按 glob 圈定,量化对比节点/边/名称覆盖率
<code>schema-index</code>	扫描 .sql CREATE TABLE 与 Java 实体注解,DbTable 节点入图

3.7 业务意图(propose → confirm 主线)

命令	作用
<code>propose</code>	业务模块证据包:图社区划分 + 作者目录共识命名,聚合代码/文档/测试事实,产出供 AI 生成候选的证据+中文提示词
<code>candidate</code>	候选审阅:list / show / review
<code>connect</code>	把 AI 候选链接桥接进确认图
<code>logic</code>	业务逻辑可信度报告(confirmed/candidate/stale/missing)
<code>business-doc</code>	已确认候选 + 图证据渲染成可读业务文档

3.8 总览

命令	作用
<code>dashboard</code>	自包含离线 HTML 管理面板:概览/业务模块/功能簇/检查/死代码/待澄清/纯度
<code>features</code>	功能区聚类(浏览入口,非权威划分)

3.9 MCP 工具(specslice-mcp)

search_graph / get_subgraph / explain_symbol / context_pack / impact / dead_code / check_drift —— 让任何 MCP 客户端(Claude、Cursor 等)直接查询证据图。

4. 多语言支持矩阵

层级	机制	语言
广度层(默认)	进程内 tree-sitter,统一通道	Rust、TypeScript、Python、Go、Java、C、C++、Swift、C#、Ruby、PHP、Kotlin
Dart	内置 analyzer sidecar(领域感知:Riverpod/Hive/导航/IAP)	Dart
精确层(可选)	SCIP overlay	Rust(rust-analyzer)、Go(scip-go)、TypeScript(scip-typescript)、Python(scip-python)
文档	Markdown / 需求 / ADR 扫描	.md .mdx .rst .adoc

每种语言的结构通道覆盖:容器(类/模块/命名空间)、可调用体、测试识别(xUnit、RSpec、PHPUnit、JUnit、#[test]、@Test…)、import 解析(PSR-4 尾对齐、JVM 后缀、require_relative、.NET 后缀…)。

5. 核心算法要点

5.1 业务模块划分与命名(propose)

这是 SpecSlice 区别于"目录罗列"式工具的核心。流程:

- 图社区发现:在文件级依赖图上做社区聚类(确定性)。
 - 成员边界约束:社区不得跨越发布单元 —— 顶层子项目(spring-core/)、工作空间容器子目录(creates/<x>、packages/<x>)。测试文件跟随其最强生产邻居。
 - 命名规则链(逐级回退,全部通用):
 - (a) 受信目录共识:特性目录 token 投票;多数派只在"实际投票者"中裁决(测试路径无受信 token,不稀释分母);清晰相对优势(≥2× 次名)也可当选;
 - (a15) 唯一成员目录:社区完全落在多成员仓库的单一子项目内时,用该子项目名(品牌前缀剥离:spring-core → core,与 token 命名的兄弟社区自然合并);
 - (a2) 支配文件:扁平布局(Redis src/)下,入边最集中的文件名即模块名(dict.c → dict);
 - (b) 中央业务符号:仅类型可命名,泛化名(load/state)与测试脚手架排除;
 - (c) 目录兜底。
 - 路径穿透(全部有真实大仓回归用例):JVM src/main/java + 反域名前缀、KMP 自定义 source set(commonJvmAndroid)、产品同名子项目保留(spring-jdbc → jdbc)、PSR-4 厂商命名空间(src/Illuminate/<x>)、合并时生产前缀恒优先于测试前缀。
 - 报告过滤:测试树模块、纯辅助路径(工具/示例/基准)不进业务模块报告。
- 实测对齐度:Spring → 官方模块表(web/core/beans/jdbc/jms/context/aop/tx);Rails → 9 个 gem 全部正确;rust-analyzer → crate 架构——对应;Laravel → Illuminate 组件级。

5.2 搜索排序

结构信号(标识符/路径/证据/邻接)与 BM25
内容信号融合;路径分类降权(tests/tools/examples/benchmarks),issue
式查询优先命中生产代码(用真实 Redis issues 验证);CJK bigram
让中文概念查询(如"错位竞争")可命中英文代码。

5.3 纯度与死代码

- 纯度:符号体内副作用模式(io/async/ui/time/randomness/global_mutation)分类,再沿调用图不动点传播。
- 死代码:从隐式入口(main/路由/测试/导出 API/框架回调,按语言与框架统一抽象为 `implicit_entry_ids`)做可达性;不可达符号附原因与置信度,公共 API 显式降低置信度。

5.4 文档↔代码漂移

文档中的路径/符号引用解析失败 → `doc_stale_code_ref`;需求无实现关联 → 孤儿需求 + 图建议实现(基于名称/路径/邻接的 hint);声明链接断裂、关联测试缺失分级报警,可配置忽略清单。

6. 质量与测试体系

- 测试驱动:新行为必须先有失败测试;当前工作区 ~1100+ 测试,60+ 测试套件。
- 分层:单元(命名规则/路径穿透/解析器)→ 集成(端到端索引多语言 fixture)→ 黄金用例(p4 pixcraft、p5 search、p7 dead-code、p9 business candidates)→ 属性测试(proptest:任意 UTF-8 输入不 panic、schema/路由管线确定性、store 往返一致)→ 实仓验收(Redis/TS/Django/gin/gson/Spring/Laravel/Rails/OkHttp/Jellyfin/rust-analyzer/bloc + 自举)。
- 零警告策略:CI 强制 `fmt + clippy -D warnings` + 全测试。
- 自举:SpecSlice 持续索引自身,6 个 crate 模块划分与工作空间一一对应,是常驻验收标准。

6.1 两轮深度审计(2026-06)

两批共 60 个审查问题(issues.md / issues2.md)逐条源码复核,全部按 TDD 闭环(先失败测试、后修复、再全量回归):

批次	确认修复	按设计(已文档化)	误报(已加回归测试钉死)	与既有修复重合
第一批 #1-#30	25	3	2	—
第二批 #31-#60	22	3	3	2

加固覆盖的代表性主题:

- 中文/Unicode 正确性:HTML 报告 UTF-8 安全转义、CJK 搜索 bigram、中文标题 slug 去塌缩、Unicode 标识符结构指纹;
- 安全与资源上限:innerHTML 注入消除、LSP 帧大小上限、glob 病态回溯 memoize、片段读取路径穿越/大小防御、BFS 节点预算、dashboard 不泄露宿主绝对路径;
- 跨语言路由管线:服务端 `{id}` / 客户端 `${id}` / Flask `<int:id>` 统一规范化、Python router 前缀传播、Go 字符串转义保真 —— 配套 Spring 服务端 ↔ Dart 客户端端到端对齐测试;

- 并发与事务:stats 账本文件锁、COMMIT 与 WAL housekeeping 失败语义分离、批量 upsert 事务化;
- 协议合规:MCP JSON-RPC 版本校验、批量请求明确拒绝、通知语义对齐规范。

7. 数据与隐私

- 全部数据落在仓库内 `.specslicelice/`(SQLite + JSONL),无任何网络上传;
- `propose` 生成的是"证据 + 提示词",把 LLM 调用留给用户自己的客户端 —— SpecSlice 本体不内置任何模型调用;
- 删除 `.specslicelice/` 即完全清除。

8. 功能审查(疏漏 / 可补充 / 可优化 / 候选裁撤)

本节是面向路线决策的中立自查,按"问题 → 建议"组织。

8.1 疏漏(已识别、未覆盖)

#	疏漏	影响	建议
1	跨服务消费边自动检测弱:HTTP 客户端调用 → 服务端路由的 consumer 边仅在部分框架命中(gin 实测 consumed routes = 0)	route-coverage 在"客户端图"侧偏空	为常见 HTTP 客户端(fetch/axios/OkHttp/request/net.http)补统一的 outbound-call 抽取
2	C#/Ruby/PHP/Kotlin 无精确层:SCIP 生态对这四语缺省	这四语只有结构边	评估 scip-dotnet / scip-ruby 成熟度;不成熟则保持"仅结构"提示
3	历史维度缺失:无 git 演化信号(热点、共改耦合)	模块边界与重构建议少一个高价值信号	可选 <code>--with-git-signals</code> :共改频率作为社区聚类的辅助边权
4	测试夹具污染模块统计:rust-analyzer <code>parser</code> 模块 539 files(大量 test_data fixture 被计为文件节点)	file_count 虚高,误导浏览	fixture 目录(<code>test_data/</code> 、 <code>testdata/</code> 、 <code>fixtures/</code>)计入辅助路径,从 file_count 剔除
5	监视模式缺失:每次手动 <code>index</code>	大仓编辑-验证循环略钝	<code>specslicelice watch</code> (fs 事件 → 增量索引);已有增量基础,工程量可控

8.2 可补充(高价值候选)

1. 语义检索层(可选):BM25 之上加本地 embedding 重排(保持零网络默认关闭),概念查询命中率再上台阶。
2. dashboard 交互深化:面板内直接发起 trace/search(目前是静态聚合);大图渲染切 Canvas/WebGL。
3. `explain-module`:propose 模块 → 单模块深挖报告(入口、对外契约、不变量、测试覆盖),衔接 business-doc。
4. CI 模板:官方 GitHub Action(index + check + impact 注释 PR),把漂移检查变成默认习惯。

8.3 可优化(实现已有、不够理想)

现状	优化方向
SCIP 首跑无进度反馈(大仓 40s+ 静默)	子进程输出透传 + 阶段提示
<code>slice / context</code> 依赖 <code>links.yaml</code> 人工维护,实际使用率低	与 <code>candidate</code> 确认流打通:确认即落 <code>links</code>
<code>features</code> 聚类与 <code>propose</code> 社区算法重叠	统一为同一套社区结果的两种视图
dead-code 的 <code>public_api_roots</code> 配置仍偏 Dart 习惯	按语言家族给默认(<code>lib.rs</code> <code>pub</code> 、 <code>__init__.py</code> 、 <code>index.ts</code>)
解析预算超时文件静默降级	<code>index --report-skipped</code> 列出被跳过文件与原因

8.4 候选裁撤(与定位不符或重叠)

候选	理由	建议
<code>schema-index</code> 独立命令	本质是索引 <code>pass</code> ,而非用户动作;Java 注解特化味重	并入 <code>index</code> 的可选 <code>enrichment</code> (配置开关),命令 保留薄别名一个版本后移除
<code>candidate / logic / business-doc</code> 三命令分立	同一工作流的三个查看角度,入口 过散	合并为 <code>specslice business</code> <code><list/show/review/doc/report></code> 子命令组
<code>stats</code>	自我观测,用户价值边缘	保留(成本趋零),但从 README 主表降级
<code>export</code>	与直接拷贝 <code>graph.db</code> 差异小	若后续无 <code>bundle</code> 消费方,降级为文档说明

8.5 明确不做(防扩散)

- 不做 IDE 插件级实时诊断(LSP 服务器):与定位(批处理证据层)冲突,生态已有成熟方案;
- 不做自动修码/自动删死代码:违背"报告不动手"承诺;
- 不内置 LLM 调用:保持零网络、零密钥;提示词与证据由用户的 Agent 消费。

9. 路线图(建议优先级)

1. P0:消费边检测补强(8.1#1)、`fixture` 污染剔除(8.1#4)、SCIP 进度反馈;
 2. P1:`watch` 模式、`business` 命令组合并、CI 模板;
 3. P2:`git` 共改信号、语义重排层、`dashboard` 交互深化;
 4. 持续:每新增一种仓库形态(语言/构建系统/目录惯例),先写失败回归测试再实现通用规则。
- 文档版本:与仓库同步演进;数字为 Apple Silicon 本机实测,随硬件浮动。